

Tema 4: PL/SQL Avanzado

1. Procedimientos Almacenados

CREATE PROCEDURE: Define un procedimiento almacenado reutilizable.

```
1 CREATE PROCEDURE nombre_proc(  
2   param1 IN tipo,  
3   param2 OUT tipo,  
4   param3 IN OUT tipo  
5 ) AS  
6   -- Declaraciones  
7   variable tipo;  
8 BEGIN  
9   -- Código del procedimiento  
10  SELECT col INTO variable FROM tabla WHERE id = param1;  
11  param2 := variable * 2;  
12 EXCEPTION  
13  WHEN NO_DATA_FOUND THEN  
14    param2 := 0;  
15 END;  
16 /
```

Parámetros:

IN: Parámetro de entrada (solo lectura).

OUT: Parámetro de salida (solo escritura).

IN OUT: Parámetro de entrada/salida.

Ejecución:

```
1 EXECUTE nombre_proc(val1, :var_out, :var_inout);  
2 -- 0 en bloque PL/SQL:  
3 BEGIN  
4   nombre_proc(10, resultado, contador);  
5 END;  
6 /
```

Eliminar procedimiento:

```
1 DROP PROCEDURE nombre_proc;
```

2. Funciones

CREATE FUNCTION: Define una función que devuelve un valor.

```
1 CREATE FUNCTION calc_bonus(  
2   salario NUMBER,  
3   años NUMBER  
4 ) RETURN NUMBER AS  
5   bonus NUMBER;  
6 BEGIN  
7   IF años > 5 THEN  
8     bonus := salario * 0.15;  
9   ELSE  
10    bonus := salario * 0.10;  
11  END IF;  
12  RETURN bonus;  
13 EXCEPTION  
14  WHEN OTHERS THEN  
15    RETURN 0;  
16 END;  
17 /
```

Uso en consultas:

```
1 SELECT nombre, salario,  
2        calc_bonus(salario, años_servicio) AS bonus  
3 FROM empleados;
```

Diferencias con procedimientos:

- Función: RETURN obligatorio, puede usarse en SELECT
- Procedimiento: Sin RETURN, solo ejecutable con EXECUTE/CALL

3. Triggers (Disparadores)

CREATE TRIGGER: Define acciones automáticas antes/después de eventos DML.

Sintaxis básica:

```
1 CREATE TRIGGER nombre_trigger  
2 {BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
3 ON tabla  
4 [FOR EACH ROW]  
5 [WHEN (condición)]  
6 DECLARE  
7   -- Declaraciones opcionales  
8 BEGIN  
9   -- Código del trigger  
10 END;  
11 /
```

Trigger de nivel de fila (FOR EACH ROW):

```
1 CREATE TRIGGER audit_salario  
2 BEFORE UPDATE OF salario ON empleados  
3 FOR EACH ROW  
4 BEGIN  
5   IF :NEW.salario < :OLD.salario THEN  
6     RAISE_APPLICATION_ERROR(-20001,  
7       'Salario no puede decrecer');  
8   END IF;  
9   INSERT INTO auditoria VALUES(  
10    :OLD.emp_id, :OLD.salario,  
11    :NEW.salario, SYSDATE);  
12 END;  
13 /
```

Variables :NEW y :OLD:

:NEW: Valores nuevos (INSERT, UPDATE).

:OLD: Valores antiguos (UPDATE, DELETE).

No disponibles en triggers de nivel de sentencia.

Trigger de nivel de sentencia:

```
1 CREATE TRIGGER log_cambios  
2 AFTER INSERT OR UPDATE OR DELETE ON tabla  
3 BEGIN  
4   INSERT INTO log VALUES(USER, SYSDATE, 'DML en tabla');  
5 END;  
6 /
```

INSTEAD OF trigger (para vistas):

```
1 CREATE TRIGGER upd_vista  
2 INSTEAD OF UPDATE ON vista_compleja  
3 FOR EACH ROW  
4 BEGIN  
5   UPDATE tabla1 SET col = :NEW.col WHERE id = :OLD.id;  
6   UPDATE tabla2 SET val = :NEW.val WHERE id = :OLD.id;  
7 END;  
8 /
```

Gestión de triggers:

```
1 -- Deshabilitar/habilitar  
2 ALTER TRIGGER nombre_trigger DISABLE;  
3 ALTER TRIGGER nombre_trigger ENABLE;  
4 -- Eliminar  
5 DROP TRIGGER nombre_trigger;
```

4. Paquetes (PACKAGES)

CREATE PACKAGE: Agrupa procedimientos, funciones, variables y tipos relacionados.

Especificación (interfaz pública):

```
1 CREATE PACKAGE pkg_empleados AS  
2   -- Constantes públicas  
3   tasa_impuesto CONSTANT NUMBER := 0.21;  
4   -- Variables públicas  
5   total_procesados NUMBER := 0;  
6   -- Funciones públicas  
7   FUNCTION calc_neto(bruto NUMBER) RETURN NUMBER;  
8   -- Procedimientos públicos  
9   PROCEDURE aumentar_salario(emp_id NUMBER, pct NUMBER);  
10 END pkg_empleados;  
11 /
```

Cuerpo (implementación):

```
1 CREATE PACKAGE BODY pkg_empleados AS  
2   -- Variables privadas  
3   contador_interno NUMBER := 0;  
4  
5   -- Función privada  
6   FUNCTION validar_empleado(id NUMBER) RETURN BOOLEAN AS  
7     v_count NUMBER;  
8   BEGIN  
9     SELECT COUNT(*) INTO v_count FROM empleados WHERE emp_id = id;  
10    RETURN v_count > 0;  
11  END;  
12  
13   -- Implementación función pública  
14   FUNCTION calc_neto(bruto NUMBER) RETURN NUMBER AS  
15   BEGIN  
16     RETURN bruto * (1 - tasa_impuesto);  
17  END;  
18  
19   -- Implementación procedimiento público  
20   PROCEDURE aumentar_salario(emp_id NUMBER, pct NUMBER) AS  
21   BEGIN  
22     IF validar_empleado(emp_id) THEN  
23       UPDATE empleados SET salario = salario * (1 + pct/100)  
24       WHERE emp_id = emp_id;  
25       total_procesados := total_procesados + 1;  
26     END IF;  
27  END;  
28 END pkg_empleados;  
29 /
```

Uso de paquetes:

```
1 -- Acceder a elementos públicos  
2 DECLARE  
3   neto NUMBER;  
4 BEGIN  
5   neto := pkg_empleados.calc_neto(5000);  
6   pkg_empleados.aumentar_salario(101, 10);  
7   DBMS_OUTPUT.PUT_LINE('Total: ' || pkg_empleados.total_procesados);  
8 END;  
9 /
```

5. Manejo de Excepciones

Estructura EXCEPTION:

```
1 DECLARE  
2   v_salario NUMBER;  
3 BEGIN  
4   SELECT salario INTO v_salario FROM empleados WHERE id = 999;  
5 EXCEPTION  
6   WHEN NO_DATA_FOUND THEN  
7     DBMS_OUTPUT.PUT_LINE('Empleado no encontrado');  
8   WHEN TOO_MANY_ROWS THEN  
9     DBMS_OUTPUT.PUT_LINE('Múltiples empleados');  
10  WHEN OTHERS THEN  
11    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  
12 END;  
13 /
```

Excepciones predefinidas:
NO.DATA.FOUND: SELECT INTO sin resultados.
TOO.MANY.ROWS: SELECT INTO con múltiples filas.
ZERO.DIVIDE: División por cero.
DUP.VAL.ON.INDEX: Violación de clave única.
INVALID.NUMBER: Conversión numérica inválida.
VALUE.ERROR: Error aritmético, conversión o truncamiento.

Excepciones definidas por usuario:

```
1 DECLARE
2   salario_bajo EXCEPTION;
3   v_salario NUMBER;
4 BEGIN
5   SELECT salario INTO v_salario FROM empleados WHERE id = 101;
6   IF v_salario < 1000 THEN
7     RAISE salario_bajo;
8   END IF;
9 EXCEPTION
10  WHEN salario_bajo THEN
11    DBMS_OUTPUT.PUT_LINE('Salario menor al mínimo');
12 END;
13 /
```

RAISE_APPLICATION_ERROR: Genera error personalizado.

```
1 IF v_edad < 18 THEN
2   RAISE_APPLICATION_ERROR(-20001,
3     'Edad debe ser >= 18');
4 END IF;
```

Rango: -20000 a -20999 para errores de usuario.
PRAGMA EXCEPTION_INIT: Asocia nombre a código de error.

```
1 DECLARE
2   fk_violation EXCEPTION;
3   PRAGMA EXCEPTION_INIT(fk_violation, -2291);
4 BEGIN
5   DELETE FROM departamentos WHERE dept_id = 10;
6 EXCEPTION
7   WHEN fk_violation THEN
8     DBMS_OUTPUT.PUT_LINE('No se puede borrar: hay empleados');
9 END;
10 /
```

6. Cursores Avanzados

Cursor explícito con parámetros:

```
1 DECLARE
2   CURSOR c_emp(dept_id NUMBER, min_sal NUMBER) IS
3     SELECT nombre, salario FROM empleados
4       WHERE departamento_id = dept_id AND salario > min_sal;
5   v_nombre empleados.nombre%TYPE;
6   v_salario empleados.salario%TYPE;
7 BEGIN
8   OPEN c_emp(10, 3000);
9   LOOP
10    FETCH c_emp INTO v_nombre, v_salario;
11    EXIT WHEN c_emp%NOTFOUND;
12    DBMS_OUTPUT.PUT_LINE(v_nombre || ': ' || v_salario);
13  END LOOP;
14  CLOSE c_emp;
15 END;
16 /
```

FOR...LOOP con cursor (más simple):

```
1 BEGIN
2   FOR rec IN (SELECT nombre, salario FROM empleados) LOOP
3     DBMS_OUTPUT.PUT_LINE(rec.nombre || ': ' || rec.salario);
4   END LOOP;
5 END;
6 /
```

Cursor con UPDATE/DELETE (FOR UPDATE):

```
1 DECLARE
2   CURSOR c_emp IS
3     SELECT emp_id, salario FROM empleados
4       WHERE departamento_id = 10
5     FOR UPDATE OF salario NOWAIT;
6 BEGIN
7   FOR rec IN c_emp LOOP
8     UPDATE empleados SET salario = salario * 1.1
9     WHERE CURRENT OF c_emp;
10  END LOOP;
11  COMMIT;
12 END;
13 /
```

Atributos de cursor:
%FOUND: TRUE si último FETCH exitoso.
%NOTFOUND: TRUE si último FETCH sin datos.
%ROWCOUNT: Número de filas procesadas.
%ISOPEN: TRUE si cursor abierto.

Cursor REF CURSOR (dinámico):

```
1 DECLARE
2   TYPE t_cursor IS REF CURSOR;
3   c_gen t_cursor;
4   v_nombre VARCHAR2(100);
5 BEGIN
6   OPEN c_gen FOR 'SELECT nombre FROM empleados WHERE dept_id = 10';
7   LOOP
8     FETCH c_gen INTO v_nombre;
9     EXIT WHEN c_gen%NOTFOUND;
10    DBMS_OUTPUT.PUT_LINE(v_nombre);
11  END LOOP;
12  CLOSE c_gen;
13 END;
14 /
```

7. Control Avanzado

CASE expresión:

```
1 v_calif := CASE
2   WHEN v_nota >= 90 THEN 'Sobresaliente'
3   WHEN v_nota >= 70 THEN 'Notable'
4   ELSE 'Aprobado' END;
```

Loop con etiquetas:

```
1 <<ext>> FOR i IN 1..10 LOOP
2   <<int>> FOR j IN 1..10 LOOP
3     EXIT ext WHEN cond;
4   END LOOP int;
5 END LOOP ext;
```

CONTINUE:

```
1 FOR i IN 1..100 LOOP
2   CONTINUE WHEN MOD(i,2)=0;
3   procesar(i);
4 END LOOP;
```

8. Colecciones

VARRAY (array tamaño fijo):

```
1 DECLARE
2   TYPE t_nums IS VARRAY(10) OF NUMBER;
3   v_nums t_nums := t_nums(1,2,3,4,5);
4 BEGIN
5   v_nums.EXTEND; v_nums(6) := 100;
6 END;
7 /
```

Nested Table (tamaño variable):

```
1 DECLARE
2   TYPE t_noms IS TABLE OF VARCHAR2(50);
3   v_noms t_noms := t_noms('Ana','Luis');
4 BEGIN
5   v_noms.EXTEND(1); v_noms(3) := 'Pedro';
6 END;
7 /
```

Associative Array (índice string/number):

```
1 DECLARE
2   TYPE t_sal IS TABLE OF NUMBER INDEX BY VARCHAR2(50);
3   v_sal t_sal;
4 BEGIN
5   v_sal('Ana') := 3000; v_sal('Luis') := 4500;
6 END;
7 /
```

Métodos: COUNT, FIRST, LAST, NEXT(i), PRIOR(i), EXISTS(i), EXTEND(n), DELETE(i), TRIM(n).

9. SQL Dinámico

EXECUTE IMMEDIATE:

```
1 DECLARE
2   v_sql VARCHAR2(1000); v_count NUMBER;
3 BEGIN
4   v_sql := 'SELECT COUNT(*) FROM empleados';
5   EXECUTE IMMEDIATE v_sql INTO v_count;
6   v_sql := 'UPDATE empleados SET salario=salario*1.1 WHERE dept=:1';
7   EXECUTE IMMEDIATE v_sql USING 10;
8 END;
9 /
```

10. Transacciones

AUTONOMOUS_TRANSACTION:

```
1 CREATE PROCEDURE log_error(msg VARCHAR2) AS
2   PRAGMA AUTONOMOUS_TRANSACTION;
3 BEGIN
4   INSERT INTO error_log VALUES(SYSDATE, msg);
5   COMMIT;
6 END;
7 /
```

SAVEPOINT:

```
1 BEGIN
2   UPDATE empleados SET salario = salario * 1.1;
3   SAVEPOINT sp1;
4   UPDATE empleados SET bonus = 500;
5   ROLLBACK TO sp1;
6   COMMIT;
7 END;
8 /
```

11. Ejemplos Prácticos

Procedimiento con cursor:

```
1 CREATE PROCEDURE ajustar_salarios(dept_id NUMBER, pct NUMBER) AS
2   CURSOR c_emp IS SELECT emp_id FROM empleados
3     WHERE departamento_id = dept_id FOR UPDATE;
4 BEGIN
5   FOR rec IN c_emp LOOP
6     UPDATE empleados SET salario = salario * (1 + pct/100)
7     WHERE CURRENT OF c_emp;
8   END LOOP;
9   COMMIT;
10 EXCEPTION
11  WHEN OTHERS THEN ROLLBACK;
12 END;
13 /
```

Trigger de auditoría:

```
1 CREATE TRIGGER audit_salary
2   AFTER UPDATE OF salario ON empleados FOR EACH ROW
3 BEGIN
4   INSERT INTO auditoria VALUES(:OLD.emp_id, :OLD.salario,
5     :NEW.salario, USER, SYSDATE);
6 END;
7 /
```